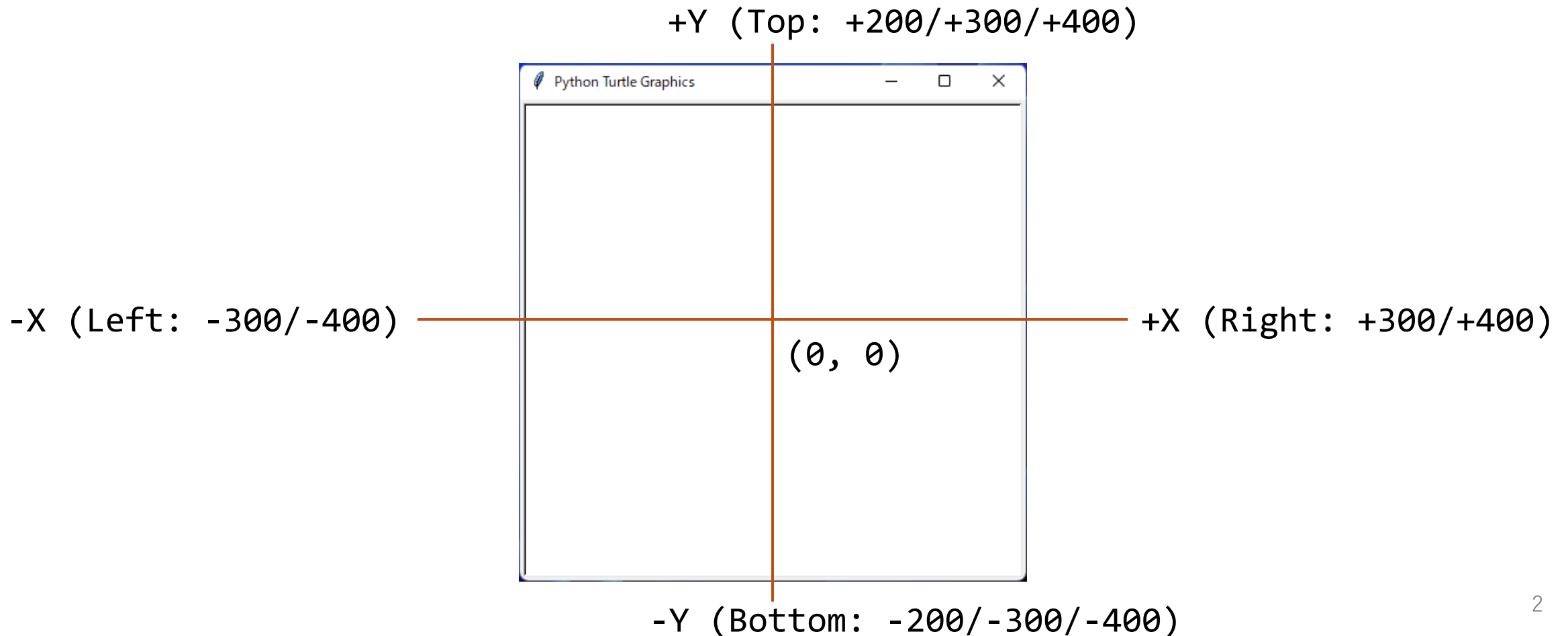


Turtle Function Cheat Sheet (for Game Development)

0. Coordinate System & Screen Boundaries

- Center Origin: The middle of the screen is always (0, 0).
- X-Axis (Horizontal): Moving Right increases X (+X), moving Left decreases X (-X).
- Y-Axis (Vertical): Moving Up increases Y (+Y), moving Down decreases Y (-Y).



1. Screen Setup & Window Management

<code>wn = turtle.Screen()</code>	Initializes the game window screen.
<code>wn.title("Game Title")</code>	Sets the title of the game window.
<code>wn.bgcolor("color")</code>	Change the background color (accepts names like "black" or hex codes like "#1a1a2e")
<code>wn.setup(width=600, height=400)</code>	Sets the dimensions of the game window.
<code>wn.tracer(0)</code>	Disables automatic screen updates. This eliminates lag and enable smooth, frame-by-frame rendering.
<code>wn.update()</code>	Manually refreshes the screen to show the latest frame (used at the end of the game loop).
<code>wn.ontimer(function, time_ms)</code>	Calls a function after a specified number of milliseconds (used to create a smooth 60 FPS game loop).

2. Object Initialization & Customization

<code>t = turtle.Turtle()</code>	Create a new turtle object.
<code>t.shape("circle")</code>	Sets the shape of the object ("square", circle, "triangle, etc.").
<code>t.color("color")</code>	Set the color of the object
<code>t.shapesize(stretch_wid=1, stretch_len=5)</code>	Stretches the object's dimensions (essential for turning a square into a long paddle).
<code>t.penup()</code>	Lifts the pen up so the object moves without drawing line across the screen.
<code>t.pendown()</code>	Put the pen down (useful if you want the ball to leave a trail or "rainbow path").
<code>t.hideturtle()</code>	Hide the turtle icon (commonly used for the pen that writes text/scores so only the text is visible).

3. Coordinates & Movement

<code>t.goto(x, y)</code>	Instantly teleports the object to the specified (x, y) coordinates.
<code>t.xcor()</code>	Returns the current X-coordinate of the object
<code>t.ycor()</code>	Returns the current Y-coordinate of the object
<code>t.setx(new_x)</code>	Moves the object to a new X-coordinate while keeping Y the same.
<code>t.sety(new_y)</code>	Moves the object to a new Y-coordinate while keeping X the same.

4. Event Handling

<code>t.distance(other_turtle)</code>	Calculates the distance (in pixels) between this object and another turtle object (the ultimate function for paddle-ball collision detection).
<code>wn.listen()</code>	Tells the window screen to start listening for keyboard inputs.
<code>wn.onkeypress(function, "key")</code>	Binds a keyboard key to a specific function (for example, <code>wn.onkeypress(move_left, "Left")</code>).

5. UI & Scoring System

```
t.write("Text", align="center", font=("Courier", 24, "bold"))
```

Displays text/scores on the screen. The example above displays “Text” at center in boldface, 24 point Courier font.

```
f"Score: {score}"
```

F-string for inserting variable (`score`) into text.

```
t.clear()
```

Erases the text written by this specific turtle in the previous frame (must be called before updating the score, otherwise new numbers will draw over old numbers).

Turtle Function Usage

1. Variables & Basic Math

<code>score = 0</code>	Creates a variable to hold the score.
<code>score += 1</code>	Increases the score by 1. (Short for <code>score = score + 1</code>)
<code>ball.dx = 4</code>	Creates a custom variable attached to the ball to control its speed in the X axis
<code>ball.dx *= -1</code>	Inverts the direction (multiplies the current speed by -1) in the X axis. This is essential for making the ball bounce!
<code>ball.dy = 5</code>	Creates a custom variable attached to the ball to control its speed in the Y axis
<code>ball.dy *= -1</code>	Inverts the direction (multiplies the current speed by -1) in the Y axis.
<code>abs(variable)</code>	Returns absolute value of <code>variable</code> . Used to check boundaries, ex: <code>abs(ball.xcolor() > 280)</code>
<code>game_active = True</code>	Boolean flag used to control if the game loop should keep running.
<code>return</code>	Exits the function (used to stop the game loop on Game Over)

2. Python Indentation Rule

Python does NOT use curly brackets `{}` to group code. It uses spaces (4 spaces or 1 Tab).

- Code inside a function or an **IF** statement must be indented

```
if abs(ball.xcor()) > 280:  
    ball.dx *= -1 # indented -> belongs to the IF statement  
print("Hello")    # NOT indented -> runs independently
```

3. Functions and **global** Keyword

Functions are blocks of code that run only when called.

- `def move_left()` – Defines a function named `move_left`
- Variables created inside a function will be automatically deleted after the function finishes running.
- If you want to modify a variable created outside the function, you must use the **global** keyword inside the function first.

```
score = 0 # a variable outside the function
def increase_score():
    global score # Requests permission to change the outside 'score'
    score += 1 # Modify the variable score inside the function
```

4. Conditional Statements (**if/else**)

Used to make decisions in the game (Ex: checking if the ball hit the wall).

- Comparison operators: **>** (greater than), **<** (less than), **==** (equal to).
- Logic operators: **or**, **and**.

```
# If the ball goes too far right OR too far left
if ball.xcor() > 290 or ball.xcor() < -290:
    ball.dx *= -1 # Reverse horizontal direction
```

- Using **abs** function

```
# If the ball goes too far right OR too far left
if abs(ball.xcor()) > 290:
    ball.dx *= -1 # Reverse horizontal direction
```

5. Lists & Randomness

Used to pick random elements, like changing colors upon collision

- Setup: You need to import the library first: `import random`
- `colors = ["red", "blue", "green"]` – Creates a list of choices
- `random.choice(colors)` – Randomly picks one item from the list.

```
# Change ball color randomly when it hits the paddle  
hit_color = random.choice(colors)  
ball.color(hit_color)
```

6. Collision Logic (Basic)

```
if ball.ycor() < -140 and abs(ball.xcor() - paddle.xcor()) < 50:  
    ball.dy *= -1    → bounce the ball off the paddle
```

To bounce the ball off the paddle, the program checks **TWO CONDITIONS SIMULTANEOUSLY**:

- **Condition 1:** `ball.ycor() < -140` (vertical threshold check)
 - The paddle is fixed at $Y = -160$ (assumption)
 - We bounce the ball only when the ball drops below $Y = -140$ (just above the paddle)
This condition prevents “phantom bounces” high in mid-air!
- **Condition 2:** `abs(ball.xcor() - paddle.xcor()) < 50` (Horizontal Distance Check)
 - The paddle is 100 pixels wide (stretching 50px left and 50px right from its center).
 - `abs()` function: Calculates the absolute horizontal distance between the ball and the paddle.
 - Purpose: Ensures the ball is aligned horizontally within the paddle's 100px body.

6. Collision Logic (Advanced)

The Issue in Basic Implementation:

- If the ball is moving upward (`ball.dy > 0`) and the player slides the paddle into it, `ball.ycor() < -140` still evaluates to True. This flips `ball.dy` back to negative, causing the ball to get stuck inside the paddle!

Solution ↓

```
if (ball.ycor() < -150 and ball.distance(paddle) < 50 and ball.dy < 0):  
    ball.dy *= -1 # Bounce the ball
```

Key Upgrades:

1. `ball.distance(paddle) < 50`: Uses built-in distance calculation to replace manual X/Y offset logic.
2. `ball.dy < 0` (Essential Guard Flag):
 - Only triggers a bounce when the ball is moving downward (`ball.dy < 0`).
 - Completely eliminates the "sticky ball" bug when moving upward!

7. Python Lists & Tuples (for TRON snake)

<code>pos = (x, y)</code>	Tuple: A fixed pair of coordinates storing X and Y together.
<code>list.append(item)</code>	Adds a new item (like current position) to the end of a list .
<code>list[:-15]</code>	List Slicing: Selects all items except the last 15 elements
<code>for _ in range(N): # process 1 # process 2 # ...</code>	Repeats a block of code N times

8. Physics & Box Collisions (for Flappy Bird)

<code>bird.dy += gravity</code>	Gravity Simulation: Gradually decreases vertical speed to pull the bird down over time
<code>bird.dy = jump_force</code>	Resets vertical speed to a positive value to instantly push the bird upward.
<code>x1 < bird.xcor() < x2</code>	Chained Comparison: Checks if the bird is inside a specific horizontal range (between pipe edges).

FORMAT OF A GAME USING TURTLE

All games follow a standardized code structure to make your learning seamless:

- **Section 1: Screen Setup**

Configures window dimensions, title, background color, tracer settings...

- **Section 2: Game Entities Initialization**

- Creates and positions `turtle.Turtle()` objects (Ball, Pipes, Boundaries).
- Sets up the score display pen.

- **Section 3: Parameters & Physics**

Defines core physics variables (speed, direction, gravity, grid sizes).

- **Section 4: Input Handling**

Binds keyboard events (`wn.onkeypress`) to action functions.

- **Section 5: Main Game Loop**

Controls continuous frame updates using `wn.ontimer(game_loop, 16)` (about 60 FPS), physics calculations, collision checks...

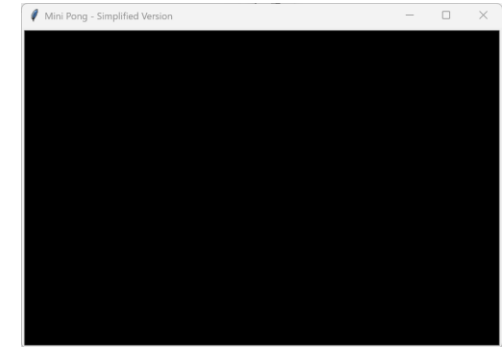
Simplified Version “mini_pong.py” step-by-step

Note:

- If you want to write a game from scratch, edit and run Python programs following step-by-step breakdown shown in the following pages.
- Refer to mini_pong.pdf (placed in the same folder with this file) for line-by-line explanations.
- If you prefer to jump straight into modifying the baseline games, you can skip this step.

Step 1 – Screen and Game Loop

```
1  import turtle
2
3  # =====
4  # SECTION 1: SCREEN SETUP
5  # =====
6  wn = turtle.Screen()
7  wn.title("Mini Pong - Simplified Version")
8  wn.bgcolor("black")
9  wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11 |
12 # =====
13 # SECTION 5: MAIN GAME LOOP (Using ontimer)
14 # =====
15 def game_loop():
16     wn.update()          # Render screen update
17     wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
18
19 # Start the game loop
20 game_loop()
21 wn.mainloop()
```



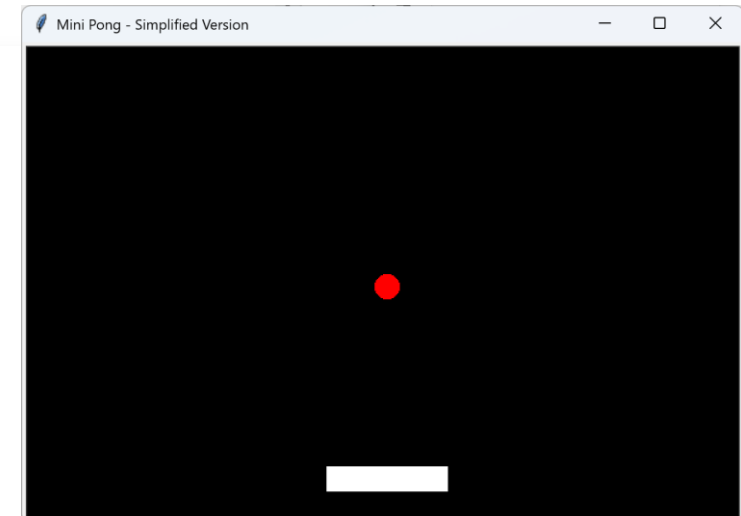
Step 2 – Adding Paddle and Ball

```
1 import turtle
2
3 # =====
4 # SECTION 1: SCREEN SETUP
5 # =====
6 wn = turtle.Screen()
7 wn.title("Mini Pong - Simplified Version")
8 wn.bgcolor("black")
9 wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11
12 # =====
13 # SECTION 2: GAME ENTITIES
14 # =====
15 # Player paddle
16 paddle = turtle.Turtle()
17 paddle.shape("square")
18 paddle.color("white")
19 paddle.shapesize(stretch_wid=1, stretch_len=5)
20 paddle.penup()
21 paddle.goto(0, -160)
```

changed from Step 1

```
22
23 # Ball entity
24 ball = turtle.Turtle()
25 ball.shape("circle")
26 ball.color("red")
27 ball.penup()
28
29 # =====
30 # SECTION 5: MAIN GAME LOOP (Using ontimer)
31 # =====
32 def game_loop():
33     wn.update() # Render screen update
34     wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
35
36 # Start the game loop
37 game_loop()
38 wn.mainloop()
```

changed from Step 1

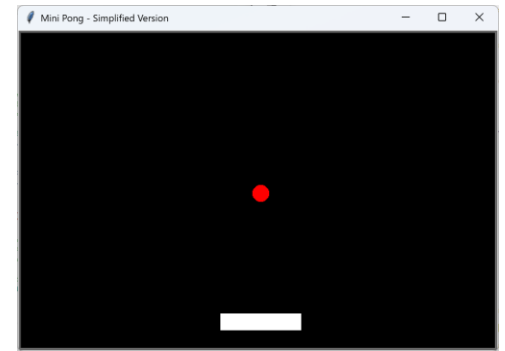


Step 3 – Move paddle upon input

```
1 import turtle
2
3 # =====
4 # SECTION 1: SCREEN SETUP
5 # =====
6 wn = turtle.Screen()
7 wn.title("Mini Pong - Simplified Version")
8 wn.bgcolor("black")
9 wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11
12 # =====
13 # SECTION 2: GAME ENTITIES
14 # =====
15 # Player paddle
16 paddle = turtle.Turtle()
17 paddle.shape("square")
18 paddle.color("white")
19 paddle.shapesize(stretch_wid=1, stretch_len=5)
20 paddle.penup()
21 paddle.goto(0, -160)
22
23 # Ball entity
24 ball = turtle.Turtle()
25 ball.shape("circle")
26 ball.color("red")
27 ball.penup()
```

changed from Step 2

```
28
29 # =====
30 # SECTION 4: INPUT HANDLING
31 # =====
32 def move_left():
33     if paddle.xcor() > -240:
34         paddle.setx(paddle.xcor() - 30)
35
36 def move_right():
37     if paddle.xcor() < 240:
38         paddle.setx(paddle.xcor() + 30)
39
40 wn.listen()
41 wn.onkeypress(move_left, "Left")
42 wn.onkeypress(move_right, "Right")
43
44 # =====
45 # SECTION 5: MAIN GAME LOOP (Using ontimer)
46 # =====
47 def game_loop():
48     wn.update() # Render screen update
49     wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
50
51 # Start the game loop
52 game_loop()
53 wn.mainloop()
```



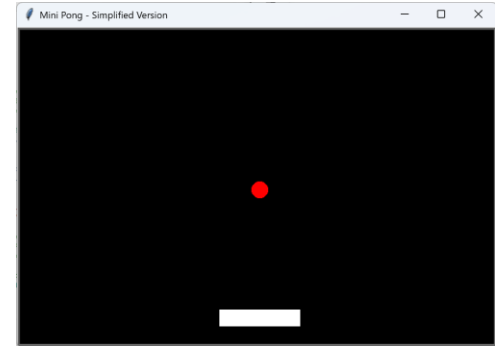
Step 4 – Move Ball every time step

```
1 import turtle
2
3 # =====
4 # SECTION 1: SCREEN SETUP
5 # =====
6 wn = turtle.Screen()
7 wn.title("Mini Pong - Simplified Version")
8 wn.bgcolor("black")
9 wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11
12 # =====
13 # SECTION 2: GAME ENTITIES
14 # =====
15 # Player paddle
16 paddle = turtle.Turtle()
17 paddle.shape("square")
18 paddle.color("white")
19 paddle.shapesize(stretch_wid=1, stretch_len=5)
20 paddle.penup()
21 paddle.goto(0, -160)
22
23 # Ball entity
24 ball = turtle.Turtle()
25 ball.shape("circle")
26 ball.color("red")
27 ball.penup()
```

changed from Step 3

```
28
29 # =====
30 # SECTION 3: PARAMETERS & PHYSICS
31 # =====
32 ball.dx = 4
33 ball.dy = 4
```

```
34
35 # =====
36 # SECTION 4: INPUT HANDLING
37 # =====
38 def move_left():
39     if paddle.xcor() > -240:
40         paddle.setx(paddle.xcor() - 30)
41
42 def move_right():
43     if paddle.xcor() < 240:
44         paddle.setx(paddle.xcor() + 30)
45
46 wn.listen()
47 wn.onkeypress(move_left, "Left")
48 wn.onkeypress(move_right, "Right")
49
50 # =====
51 # SECTION 5: MAIN GAME LOOP (Using ontimer)
52 # =====
53 def game_loop():
54     # Move ball
55     ball.setx(ball.xcor() + ball.dx)
56     ball.sety(ball.ycor() + ball.dy)
57
58     wn.update() # Render screen update
59     wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
60
61 # Start the game loop
62 game_loop()
63 wn.mainloop()
```



changed from Step 3

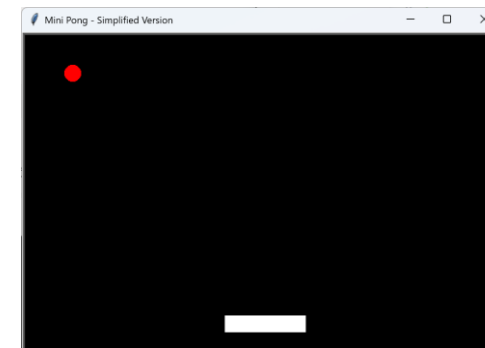
Step 5 – Adding Collision Logic

```
1 import turtle
2
3 # =====
4 # SECTION 1: SCREEN SETUP
5 # =====
6 wn = turtle.Screen()
7 wn.title("Mini Pong - Simplified Version")
8 wn.bgcolor("black")
9 wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11
12 # =====
13 # SECTION 2: GAME ENTITIES
14 # =====
15 # Player paddle
16 paddle = turtle.Turtle()
17 paddle.shape("square")
18 paddle.color("white")
19 paddle.shapesize(stretch_wid=1, stretch_len=5)
20 paddle.penup()
21 paddle.goto(0, -160)
22
23 # Ball entity
24 ball = turtle.Turtle()
25 ball.shape("circle")
26 ball.color("red")
27 ball.penup()
```

```
28
29 # =====
30 # SECTION 3: PARAMETERS & PHYSICS
31 # =====
32 ball.dx = 4
33 ball.dy = 4
34
35 # =====
36 # SECTION 4: INPUT HANDLING
37 # =====
38 def move_left():
39     if paddle.xcor() > -240:
40         paddle.setx(paddle.xcor() - 30)
41
42 def move_right():
43     if paddle.xcor() < 240:
44         paddle.setx(paddle.xcor() + 30)
45
46 wn.listen()
47 wn.onkeypress(move_left, "Left")
48 wn.onkeypress(move_right, "Right")
49
```

```
50 # =====
51 # SECTION 5: MAIN GAME LOOP (Using ontimer)
52 # =====
53 def game_loop():
54     # Move ball
55     ball.setx(ball.xcor() + ball.dx)
56     ball.sety(ball.ycor() + ball.dy)
57
58     # Wall bounce (Left / Right / Top)
59     if abs(ball.xcor()) > 280:
60         ball.dx *= -1
61     if ball.ycor() > 180:
62         ball.dy *= -1
63
64     # Paddle collision
65     if ball.ycor() < -140 and abs(ball.xcor() - paddle.xcor()) < 50 and ball.dy < 0:
66         ball.dy *= -1
67
68     wn.update() # Render screen update
69     wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
70
71 # Start the game loop
72 game_loop()
73 wn.mainloop()
```

changed from Step 4



Full program – Reset Ball if falls out of screen

```
1 import turtle
2
3 # =====
4 # SECTION 1: SCREEN SETUP
5 # =====
6 wn = turtle.Screen()
7 wn.title("Mini Pong - Simplified Version")
8 wn.bgcolor("black")
9 wn.setup(width=600, height=400)
10 wn.tracer(0) # Turn off auto-rendering for smooth gameplay
11
12 # =====
13 # SECTION 2: GAME ENTITIES
14 # =====
15 # Player paddle
16 paddle = turtle.Turtle()
17 paddle.shape("square")
18 paddle.color("white")
19 paddle.shapesize(stretch_wid=1, stretch_len=5)
20 paddle.penup()
21 paddle.goto(0, -160)
22
23 # Ball entity
24 ball = turtle.Turtle()
25 ball.shape("circle")
26 ball.color("red")
27 ball.penup()
```

```
28 # =====
29 # SECTION 3: PARAMETERS & PHYSICS
30 # =====
31 ball.dx = 4
32 ball.dy = 4
33
34 # =====
35 # SECTION 4: INPUT HANDLING
36 # =====
37
38 def move_left():
39     if paddle.xcor() > -240:
40         paddle.setx(paddle.xcor() - 30)
41
42 def move_right():
43     if paddle.xcor() < 240:
44         paddle.setx(paddle.xcor() + 30)
45
46 wn.listen()
47 wn.onkeypress(move_left, "Left")
48 wn.onkeypress(move_right, "Right")
49
50 # =====
51 # SECTION 5: MAIN GAME LOOP (Using ontimer)
52 # =====
53 def game_loop():
54     # Move ball
55     ball.setx(ball.xcor() + ball.dx)
56     ball.sety(ball.ycor() + ball.dy)
```

```
57
58 # Wall bounce (Left / Right / Top)
59 if abs(ball.xcor()) > 280:
60     ball.dx *= -1
61 if ball.ycor() > 180:
62     ball.dy *= -1
63
64 # Paddle collision
65 if ball.ycor() < -140 and abs(ball.xcor() - paddle.xcor()) < 50 and ball.dy < 0:
66     ball.dy *= -1
67
68 # Bottom boundary (Reset ball)
69 if ball.ycor() < -190:
70     ball.goto(0, 0)
71     ball.dy = 4
72
73 wn.update() # Render screen update
74 wn.ontimer(game_loop, 16) # Call game_loop again after 16ms (~60 FPS)
75
76 # Start the game loop
77 game_loop()
78 wn.mainloop()
```

changed from Step 5

